interrupt the cascade of failures. This solves both disadvantages mentioned in the initial design.

The file watcher extends `FileListeningAbilities`, which takes care of the specifics of a file-listening API, such as listening to a directory and registering the files in it. This provides more freedom in composition, code isolation, and testability.

Another important feature is that the file watcher restarts and creates a new instance of the actor if file listening fails. However, if another exception occurs, it stops not only itself but also its children: that is, its child log processor and its grandchild database writer.

Listing 5.4 File watcher sketch implementation

```
object FileWatcher extends FileListeningAbilities {

  sealed trait Command
  final case class NewFile(file: File, timeAdded: Long) extends Command
  final case class FileModified(file: File, timeAdded: Long) extends Command

  def apply(directory: String): Behavior[Command] =
    Behaviors
      .supervise {
        Behaviors.setup[Command] { context =>
          ...                                                          ❶
          ...                                                          ❷
          ...                                                          ❸
          Behaviors
            .receiveMessage[Command] {
              case NewFile(file, _) => ...
              ...                                                      ❹
              case FileModified(file, _) => ...
              ...                                                      ❺
            }
            .receiveSignal {
              case (_, Terminated(ref)) => ...
              ...                                                      ❻
            }
        }
      }
```